

# 实验报告

课程名称: Design pattern and software architecture

学 院: 计算机科学与工程学院

专 业: Computer Science and 班 级: SE-1          

Technology

姓 名: Muhammad Samudera 学 号:           

年 月 日

山东科技大学教务处制

# 实 验 报 告

\_\_\_\_\_页

|        |                                                                                                                  |     |           |       |      |
|--------|------------------------------------------------------------------------------------------------------------------|-----|-----------|-------|------|
| 组 别    | 1                                                                                                                | 姓 名 | Samudera  | 同组实验者 | None |
| 实验项目名称 | Builder Pattern                                                                                                  |     |           |       |      |
| 教师评语   | Good understanding of the pattern idea and implementation.<br>The report is clear and written in proper English. |     |           |       |      |
| 实验成绩:  |                                                                                                                  |     | 指导教师（签名）： |       |      |
|        |                                                                                                                  |     | 2026 3 23 |       |      |

## 1. Objective

The objective of this experiment is to implement the Builder design pattern using a food-ordering example involving Pizza and Burger. The goal of this pattern is to separate the construction of complex objects from their representation, allowing the same construction process to produce different results.

In this example, the program has two independent product families:

- Pizza — constructed with dough, sauce, and topping.
- Burger — constructed with bun, patty, and salad.

Each builder is responsible for assembling one type of product step by step. A Director (Waiter) orchestrates the construction process using a given builder.

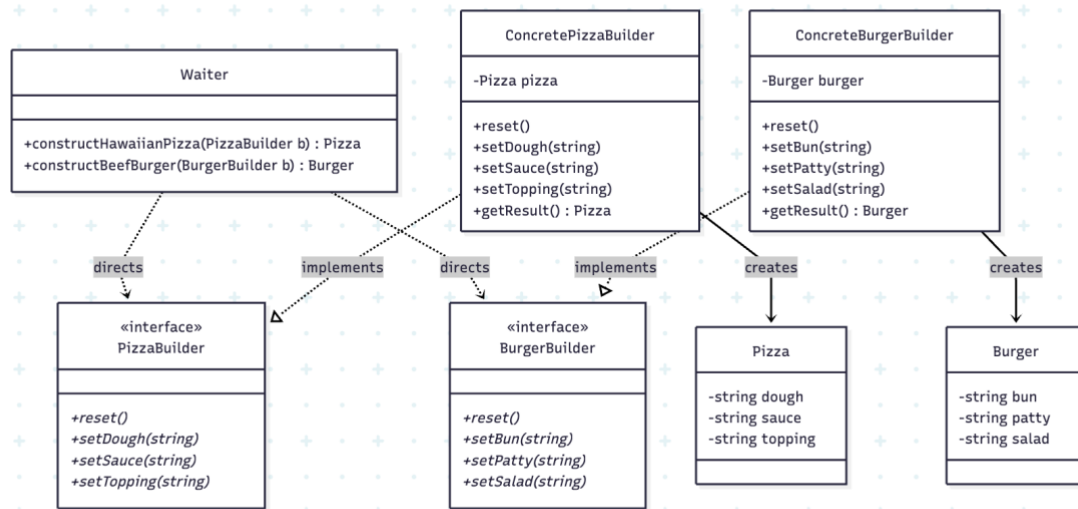
## 2. Principle

The Builder pattern separates an object's construction from its final representation. Instead of constructing a complex object directly, the client delegates construction steps to a Builder object. A Director controls the sequence of steps, while the Builder handles how each step is carried out.

Key benefits of this separation include:

- The same construction process can create different product representations.
- Construction details are hidden from the client.
- It is easy to add new builders without modifying existing code.

## 2. UML Diagram



### 3. Key Code (Java)

The following is the core implementation of the **Builder** Pattern, written as a single **Java** file.

**Builder Interfaces:**

```

interface PizzaBuilder {

    void reset();

    void setDough(String dough);

    void setSauce(String sauce);

    void setTopping(String topping);

}
  
```

```

interface BurgerBuilder {
  
```

```

    void reset();
  
```

```

    void setBun(String bun);

    void setPatty(String patty);

    void setSalad(String salad);
}

```

Concrete Builder — Pizza:

```

class ConcretePizzaBuilder implements PizzaBuilder {

    private Pizza pizza;

    public ConcretePizzaBuilder() { reset(); }

    public void reset()                { this.pizza = new Pizza(); }

    public void setDough(String dough)  { pizza.setDough(dough); }

    public void setSauce(String sauce)  { pizza.setSauce(sauce); }

    public void setTopping(String topping) { pizza.setTopping(topping); }

    public Pizza getResult() {

        Pizza result = this.pizza; reset(); return result;

    }

}

```

Director (Waiter):

```

class Waiter {

    public Pizza constructHawaiianPizza(ConcretePizzaBuilder b) {

        b.reset();

        b.setDough("thin crust");

        b.setSauce("tomato");
    }
}

```

```

        b.setTopping("ham and pineapple");

        return b.getResult();

    }

    public Burger constructBeefBurger(ConcreteBurgerBuilder b) {

        b.reset();

        b.setBun("sesame bun");

        b.setPatty("beef patty");

        b.setSalad("lettuce, tomato, onion");

        return b.getResult();

    }

}

```

#### 4. Analysis

**Advantages:** The Builder pattern provides a clear separation between object construction and its final representation. It allows the same Director to produce different objects simply by passing a different builder. The reset() method also ensures that each builder can be reused cleanly after producing a result, without needing to instantiate a new builder object.

**Disadvantage:** The pattern requires creating a separate builder class for each product type. If the number of product types grows significantly, the number of classes increases accordingly, which can add complexity to the codebase.

#### 6. Conclusion

The Builder pattern is well-suited for situations where an object needs to be constructed through multiple steps, and where different configurations of that object are needed. In this program, the Waiter (Director) does not construct objects directly; instead, it delegates each step to the appropriate builder, which keeps the construction logic organized and reusable.

This approach cleanly separates the roles of construction and representation. The client code remains independent from the internal structure of the products, which makes the design easier to maintain and extend. Adding a new food type, for example, only requires creating a new builder that implements the corresponding interface, without modifying the Director or any existing builder.

Overall, the Builder pattern is a practical and scalable solution for constructing complex objects step by step. It improves code readability and ensures that each object is assembled in a consistent and controlled manner

